# Implementation Report

Neural Collapse & Relative Flatness in Deep Learning Generalization
`github.com/TrustworthyMachineLearning-Lab/grokking_flatness`

**Abstract**

This report documents the implementation of the codebase accompanying the NeurIPS 2025 paper *"Flatness is Necessary, Neural Collapse is Not: Rethinking Generalization via Grokking"*. The repository investigates the roles of **Neural Collapse (NC)** and **Relative Flatness** in deep learning generalization, leveraging the grokking phenomenon as a controlled testbed. We describe the architecture, core modules, metric computations, experiment pipelines, and reproducibility details for all six experiment families.

## Contents

# 1 Project Overview

The codebase (`grokking_flatness`) empirically tests two central claims:

1. **Neural Collapse is not necessary** for generalization — generalization can occur without the collapse of class means to the vertices of a simplex equiangular tight frame (ETF).
2. **Relative Flatness is potentially necessary** for generalization — the phase transition from memorization to generalization in grokking coincides with a sharp drop in the sharpness measure.
3. Neural Collapse *provably leads to* Relative Flatness under classical assumptions, establishing a theoretical link between the two phenomena.

The repository contains six self-contained experiment directories plus a shared library for the grokking testbed, as summarised in Table 1.

| Directory | Purpose | Figures |
|---|---|---|
| `grokking_nc_rf/` | Grokking testbed: NC & flatness metrics | Fig. 1 |
| `nc_experiments/` | Neural Collapse clustering experiments | Fig. 2 |
| `rf_resnet18/` | Relative Flatness on ResNet-18 | Fig. 4 |
| `rf_vit/` | Relative Flatness on Vision Transformer | Fig. 4 |
| `rf_bert/` | Relative Flatness on BERT | Figs. 16,17 |
| `rf_gpt2/` | Relative Flatness on GPT-2 | Figs. 18,19 |

Table 1: Experiment directories and their corresponding paper figures.

### Dependencies

- `PyTorch 2.4.1, torchvision 0.19.1, timm 1.0.15`
- `pytorch-lightning 1.2.10` (grokking experiments only)
- `scikit-learn 1.3.2, numpy, matplotlib, scipy, tqdm`
- `transformers, datasets` (BERT / GPT-2 experiments)
- `mod` (modular arithmetic), `blobfile`, `sympy`

# 2 Core Library: `grok/`

The `grokking_nc_rf/grok/` package is the central shared library, implementing the grokking setup from Power et al. (2022) [1] with extensions for neural collapse and flatness measurement.

## 2.1 Data Module (`data.py`)

Generates algorithmic datasets for modular arithmetic and permutation groups.

`ArithmeticTokenizer` Maps token strings to integer IDs. Supports operators listed in `VALID_OPERATORS` (addition, subtraction, multiplication, division, polynomial forms over $\mathbb{Z}_{97}$, symmetric group $S_5$, list operations).

`ArithmeticDataset` Produces training/validation splits controlled by `train_data_pct` and `actual_train_data`. The "grokking gap" is created by making `actual_train_data_pct` $<$ `train_data_pct`, so the model sees only a subset of the training equations per epoch, delaying generalization.

`ArithmeticIterator` Batched PyTorch `IterableDataset` with configurable batch size, device placement, and shuffling.

### 2.2 Transformer Architecture (`transformer.py`)

A decoder-only transformer with noise-injection capabilities:

- **Noisy layers**: `Linear`, `LayerNorm`, and `Embedding` support `weight_noise` — additive Gaussian noise with variance $\sigma^2$ applied during training. This is a key knob for controlling the sharpness of the loss landscape.
- **Attention**: Single-head (`AttentionHead`) and multi-head (`MultiHeadAttention`) with causal masking, scaled dot-product attention.
- **Feed-forward**: Two-layer MLP with ReLU/GELU non-linearity and dropout.
- **Decoder block**: Pre-norm residual stream: LayerNorm($x$+Attn($x$)) followed by LayerNorm($x$+FFN($x$)).
- **Positional encoding**: Sinusoidal fixed embeddings.
- **Output head**: Linear projection to vocabulary size.

Default configuration: 2 layers, 4 heads, embedding dimension 128, context length 50.

### 2.3 Training Loop (`training.py`)

The `TrainableTransformer` class extends `LightningModule` from PyTorch Lightning.

#### Key Hyper-parameters

- `batchsize`, `n_layers`, `n_heads`, `d_model`, `dropout`
- `weight_noise`: standard deviation of injected parameter noise
- `math_operator`: arithmetic operation (e.g., '+', '-', '*')
- `train_data_pct`, `actual_train_data_pct`: control grokking gap
- `max_lr`, `anneal_lr`: cosine annealing with linear warmup
- `weight_decay_kind`: `to_zero`, `to_init`, `jiggle`, `honest`
- `opt_type`: `AdamW` (custom implementation) or `SGD`
- `use_rglr`, `reg_step`, `hessian_coeff`: optional sharpness-based regularization

#### Custom Optimizers

- `CustomAdamW`: AdamW with multiple weight-decay forms — decaying toward zero, toward initialization, or with multiplicative log-normal "jiggle" noise. Also supports adding uniform Gaussian noise to updates ('noise_factor').
- `SAM`: Sharpness-Aware Minimization (Foret et al. 2020) two-step optimizer.

#### Sharpness & Hessian Computation (inline)

During both training and validation, the model computes per-sample sharpness:

$$\mathbf{p}_i = \text{softmax}(\mathbf{z}_i)$$

$$H_i^{(1)} = \sum_{k=1}^{K} p_{i,k}(1 - p_{i,k}) \tag{1}$$

$$H_i^{(2)} = \|\phi_i\|_2^2 \tag{2}$$

$$\mathcal{H}_i = H_i^{(1)} \cdot H_i^{(2)} \tag{3}$$

$$S_i^{(1)} = \|W_{\text{out}}\|_2 \cdot \mathcal{H}_i \tag{4}$$

$$S_i^{(2)} = \|W_{\text{out}}\|_2^2 \cdot \mathcal{H}_i \tag{5}$$

where $\phi_i$ are the penultimate-layer representations and $W_{\text{out}}$ is the final linear classifier weight matrix. These are computed with `torch.no_grad()` and stored per-epoch for both train and validation sets.

### 2.4 Metrics Module (`metrics.py`)

Implements norm-based generalization measures from Neyshabur et al. [3] and Bartlett et al. [4].

`compute_measure` Recursively aggregates per-layer measures over all `nn.Linear` modules using operators `log_product`, `sum`, `max`, `product`, `norm`.

`norm` $L_{p,q}$ norm of weight matrix: $(\sum_j(\sum_i |W_{ij}|^p)^{q/p})^{1/q}$.

`op_norm` Spectral norm (largest singular value) via SVD, or $p$-Schatten norm.

`dist` $L_{p,q}$ distance from initialization.

`h_dist` Hidden-unit normalized distance: $h^{1-1/q} \cdot \text{dist}$.

`h_dist_op_norm` Ratio h_dist/op_norm.

The `calculate()` function returns a dict of measures and generalization bounds:

- **Measures**: $L_{1,\infty}$, Frobenius, $L_{3,1.5}$, Spectral, $L_{1.5}$ operator, Trace norms.
- **Bounds**: L1-Max Bound (Bartlett 2002), Frobenius Bound (Neyshabur 2015), $L_{3,1.5}$ Bound, Spectral $L_{2,1}$ Bound (Bartlett 2017), Spectral-Frobenius Bound (Neyshabur 2018).

### 2.5 Sharpness Computation (`measure.py`)

Implements the $C_{\text{sharpness}}$ measure from Keskar et al. [2]:

1. Flatten model parameters into vector $\mathbf{x}_0$.
2. Project onto a random subspace of dimension $d$ (default $d = 10$) via random matrix $\mathbf{A}_+ \in \mathbb{R}^{d \times P}$ with orthonormal rows.
3. Solve constrained maximisation using L-BFGS-B:

$$\max_{\mathbf{y} \in \mathbb{R}^d} \mathcal{L}(\mathbf{x}_0 + \mathbf{A}\mathbf{y}) \quad \text{s.t.} \quad |y_i| \leq \varepsilon(|\mathbf{A}_{+,i:}\mathbf{x}_0| + 1)$$

4. Sharpness: $\phi = \frac{f_{\max} - f_0}{1 + f_0} \times 100$.

### 2.6 Visualization (`visualization.py`)

Produces multi-panel line plots of loss/accuracy vs. epochs across training-data-percentage sweeps, with colorbars indicating the percentage of training data.

- `load_metric_data`: reads CSV logs into tensors.
- `add_metric_graph`: plots one metric with color scale by $T$ (training data %).
- `add_extremum_graph`: max/min value vs. $T$.
- `find_inflections`: detects phase-transition points via moving-average sign changes.

### 2.7 Scripts

- `scripts/make_data.py`: Generates token and dataset files.
- `scripts/train.py`: Entry point for training; saves checkpoints and metrics.
- `scripts/compute_sharpness.py`: Loads checkpoints and runs Keskar sharpness.
- `scripts/create_partial_metrics.py`: Evaluate checkpoints at specified epochs.
- `scripts/create_metrics_for_epochs.py`: Batches partial metric extraction.
- `scripts/visualize_metrics.py`: Full visualization pipeline with ffmpeg video output.

## 3 Neural Collapse Experiments (`nc_experiments/`)

These experiments test whether Neural Collapse is necessary for generalization on CIFAR-10.

### 3.1 Architectures

- `resnet18.py`: ResNet-18 implementation with BasicBlock from scratch.
- `resnet18_torchvision.py`: TorchVision pre-trained ResNet-18 wrapper.

### 3.2 Metrics

The key NC metric (Equation 4 in paper) is the *within-class to between-class variance ratio*:

$$\mathrm{NC}_c = \frac{\mathrm{Var}_{\text{within}}}{\mathrm{Var}_{\text{between}}} = \frac{1}{K}\sum_{k=1}^{K}\frac{\sigma_k^2}{\|\mu_k - \mu\|^2} \tag{6}$$

Implementation in `training_utils.py`:

1. Group penultimate-layer representations by true class label.
2. For each pair of classes $(c_1, c_2)$, compute within-class variance $\sigma_{c_1}^2, \sigma_{c_2}^2$ and between-class distance $\|\mu_{c_1} - \mu_{c_2}\|^2$.
3. The cluster score for that pair is $(\sigma_{c_1}^2 + \sigma_{c_2}^2)/(2\|\mu_{c_1} - \mu_{c_2}\|^2)$.
4. The overall NC score is the mean over all class pairs.

### 3.3 Regularisation Experiments

Two types of regularisation are implemented:

1. **NC-based** (`use_pow=1`): Maximises the NC cluster score (i.e., pushes representations *away* from NC) by adding $-\lambda \cdot \mathrm{NC}_{\mathrm{score}}$ to the loss.
2. **Flatness-based** (`use_pow=0`): Minimises the sharpness $S^{(2)}$ by adding $-\lambda \cdot \mathbb{E}[S^{(2)}]$ to the loss.

For both variants, the regularisation is "cancelled" at a specified epoch, so the model undergoes two phases: forced anti-NC / anti-flatness, then free fine-tuning.

### 3.4 Running

```
cd nc_experiments
bash run_script.sh   # (edit hyperparameters inside the script)
```

## 4 Relative Flatness Experiments: ResNet-18 & ViT & BERT & GPT-2

These experiments validate the flatness hypothesis across four standard architectures.

### 4.1 Shared Design

All follow the same pattern:

1. Train with a regulariser that either **minimises** or **maximises** relative flatness (controlled by `use_reglation` and `lambda` sign).
2. After $N_{\mathrm{cancel}}$ epochs, disable the regulariser and optionally switch to weight decay + cosine annealing.
3. Track validation accuracy as a function of flatness.

### 4.2 Sharpness Definition

All architectures compute the same per-sample sharpness as the grokking experiments (Eqs. 1–5), using the final linear layer's weight norm and the penultimate-layer representation norm.

### 4.3 Per-Architecture Details

`rf_resnet18/`

- Models: ResNet-18, Mobile ResNet (MResNet), WideResNet-16-8, EfficientNet-B0/V2.
- Dataset: CIFAR-10.
- Loss functions: cross-entropy, ramp loss, squared hinge, logistic hinge, leaky ramp.
- Weight-capping: Frobenius or spectral norm constraints on classifier weights.
- Regulariser cancellation: at `cancel_epoch`, weight decay turns on and learning rate bumps by $10\times$, then cosine annealing.

`rf_vit/`

- Dataset: ImageNet-100 (subset of ImageNet with 100 classes).
- Model: ViT-Base/16 via `timm`.
- Key metrics: per-epoch sharpness, NC cluster score, train/val loss and accuracy.

`rf_bert/`

- Dataset: SST-5 (Stanford Sentiment Treebank, 5 classes).
- Model: `bert-base-uncased` via HuggingFace `transformers`.
- Penultimate layer: CLS token hidden state (index 0 of last hidden layer).
- Output head: `model.classifier.weight`.

`rf_gpt2/`

- Dataset: SST-5 (same split as BERT).
- Model: `gpt2` via HuggingFace `transformers`.
- Penultimate layer: last non-padding token hidden state.
- Output head: `model.score.weight`.

### 4.4 Training Utilities

All architectures share a common set of helper functions:

`compute_cluster` Same NC metric as in `nc_experiments`.

`ortho_rows_fro` Encourages orthonormality of classifier rows: $\|WW^\top - I\|_F^2$, scaled per-entry or per-row.

`frobenius_cap` Projects weights onto Frobenius-norm ball: if $\|W\|_F > \text{max\_norm}$, $W \leftarrow W \cdot \frac{\text{max\_norm}}{\|W\|_F}$.

`spectral_cap` Projects weights onto spectral-norm ball via power iteration.

`set_lr`, `mul_lr`, `set_wd` In-place learning rate and weight decay manipulation.

## 5 Reproducibility Guide

### 5.1 Quick Start

```
# 1. Clone
git clone https://github.com/TrustworthyMachineLearning-Lab/
    grokking_flatness.git
cd grokking_flatness

# 2. Install
pip install -r requirements.txt
pip install torch==1.13.0 pytorch-lightning==1.2.10
cd grokking_nc_rf && pip install -e . && cd ..

# 3. Generate data (grokking experiments)
cd grokking_nc_rf && python scripts/make_data.py && cd ..

```

```
# 4. Run an experiment (e.g., ResNet-18 flatness)
cd rf_resnet18
bash run_script.sh   # edit hyperparameters inside
```

### 5.2 Plotting Results

- Grokking metrics: Copy `read_loss_acc.ipynb` and `plot_grok_cluster.ipynb` into the results directory and run.
- Relative flatness results: Use `relative_flatness_plot_results.ipynb`.
- NC results: Use the saved `.npy` arrays in the experiment output directory.

### 5.3 Reproducing Paper Figures

| Figure | Script/Directory | Key Parameter(s) |
|---|---|---|
| Fig. 1 (grokking) | `grokking_nc_rf/` | `train_data_pct=20..50` |
| Fig. 2 (NC) | `nc_experiments/` | `use_pow=1, cancel_epoch=60` |
| Fig. 4 (flatness) | `rf_resnet18/, rf_vit/` | `use_reglation=1, lambda=1e-2` |
| Figs. 16,17 (BERT) | `rf_bert/bert_sst5.py` | `cancel_epoch, lambda` |
| Figs. 18,19 (GPT-2) | `rf_gpt2/gpt_sst5.py` | `cancel_epoch, lambda` |

## 6 Summary of Key Implementation Details

### 6.1 Sharpness Approximation

The sharpness metric used throughout the paper is a simplified, tractable approximation of the Hessian's trace:

$$\mathbb{E}_i \left[ \underbrace{\left(\mathbf{p}_i^\top (\mathbf{1} - \mathbf{p}_i)\right)}_{\text{probabilistic curvature}} \cdot \underbrace{\|\phi_i\|_2^2}_{\text{representation norm}} \right] \cdot \|W_{\text{out}}\|_2^p \qquad p \in \{1, 2\} \tag{7}$$

This decomposes into three interpretable factors: the model's softmax confidence (which vanishes when the model is certain), the scale of penultimate-layer activations, and the norm of the classifier weights.

### 6.2 Neural Collapse Metric

The NC score measures the ratio of within-class to between-class variance:

$$\text{NC} = \frac{1}{K(K-1)} \sum_{i \neq j} \frac{\sigma_i^2 + \sigma_j^2}{2\|\mu_i - \mu_j\|_2^2}$$

where $\mu_k$ and $\sigma_k^2$ are the empirical mean and variance of penultimate-layer representations for class $k$. Lower values indicate stronger collapse.

### 6.3 Regulariser Cancellation Protocol

A key experimental design is the "two-phase" training: the model is first regularised to suppress either NC or flatness, then at a prescribed epoch the regulariser is removed and the model is allowed to converge naturally. This isolates the causal effect of each property on downstream generalisation.

## 7 File Manifest

| File / Directory | Purpose |
|---|---|
| `README.md` | Top-level documentation and citation info |
| `requirements.txt` | Python dependencies |
| `LICENSE` | License file |
| `figure/` | Paper figures |
| **Shared Grokking Library** | |
| `grokking_nc_rf/grok/` | Core package |
| `data.py` | Algorithmic dataset generation |
| `transformer.py` | Decoder-only transformer with noise |
| `training.py` | LightningModule, CustomAdamW, SAM |
| `metrics.py` | Norm-based measures & bounds |
| `measure.py` | Keskar sharpness (L-BFGS-B) |
| `visualization.py` | Plotting utilities |
| `__init__.py` | Package init |
| `grokking_nc_rf/scripts/` | Training, data, metric, plot scripts |
| `grokking_nc_rf/setup.py` | Package install |
| **NC Experiments** | |
| `nc_experiments/resnet18.py` | ResNet-18 from scratch |
| `nc_experiments/resnet18_torchvision.py` | TorchVision ResNet-18 |
| `nc_experiments/train.py` | NC experiment entry point |
| `nc_experiments/training_utils.py` | NC metric & training loop |
| `nc_experiments/utils.py` | Data loading & plotting |
| **Relative Flatness Experiments** | |
| `rf_resnet18/` | ResNet-18 & variants |
| `rf_vit/train_vit.py` | ViT-Base/16 on ImageNet-100 |
| `rf_bert/bert_sst5.py` | BERT on SST-5 |
| `rf_gpt2/gpt_sst5.py` | GPT-2 on SST-5 |

## 8 Conclusion

The implementation is modular, with a clear separation between the shared grokking library and architecture-specific experiment scripts. Each experiment directory is self-contained with its own `run_script.sh` and supports comprehensive metric logging. The code is designed for reproducibility: all random seeds are fixed, hyperparameters are fully configurable via command-line arguments, and results are saved in structured formats (CSV logs for Lightning, NumPy arrays for per-epoch metrics, PyTorch checkpoints for model weights).

## References

[1] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*. ICLR Blog Track, 2022.

[2] N. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. Tang. *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*. ICLR, 2017.

[3] B. Neyshabur, R. Tomioaka, and N. Srebro. *Towards Generalization in Deep Learning*. NeurIPS, 2015.

[4] P. Bartlett, D. Foster, and M. Telgarsky. *Spectrally-normalized margin bounds for neural networks*. NeurIPS, 2017.

[5] V. Papyan, X. Han, and D. Donoho. *Prevalence of neural collapse during the terminal phase of deep learning training*. PNAS, 2020.

[6] T. Han, L. Adilova, H. Petzka, J. Kleesiek, and M. Kamp. *Flatness is Necessary, Neural Collapse is Not: Rethinking Generalization via Grokking*. NeurIPS, 2025.